LIÈGE université
Sciences Appliquées

AI-DJ: Spotify Hit Predictor

Presented by : Antoine Paulis
Charlotte Michon
Mohamed-Khalil Ankri

LIÈGE université
Sciences Appliquées

Presentation Outline

I. Milestone 1 + improvement
II. Code testing
III. Model pipeline
IV. Model serving
V. Model deployment

Milestone 1 + improvement

Milestone 1 + improvement

- **Milestone 1 recap :**
  - **Purpose :** Tool to predict if a song will be a “hit” or a “flop”, by training on a Spotify dataset (*kaggle*) of different genres of music.
  - **ML model :** random forest for a binary classification problem.
  - **MLOps Automation :** Gitflow on Github and Google Cloud.
- **Google Cloud improvement**:
  - ~~**Bucket**~~ **=> BigQuerry :** tabular data + better querrying and scalability.
- **Training script improvement:**

Milestone 1

Training
- EDA.ipynb
- evaluate.py
- gb.py
- knn.py
- rf.py

Milestone 2

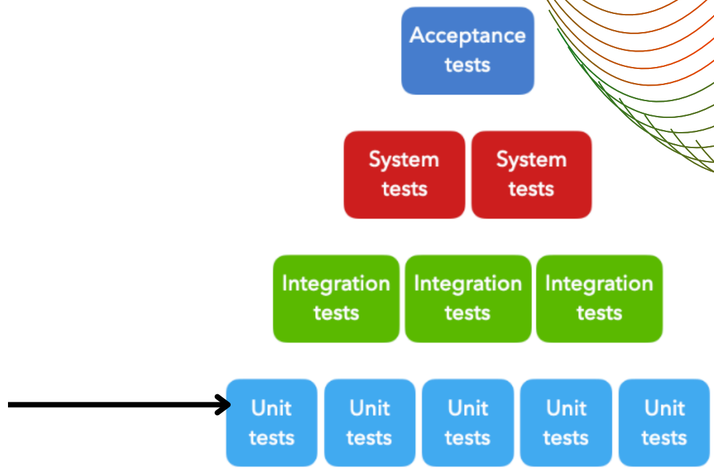
Training
- EDA.ipynb
- evaluate.py
- gb.py
- importDataset.py
- knn.py
- rf.py
- train.py
- trainingPreprocess.py

Code testing

Code testing

- **PyTests:** catch bug early, improve code quality and documentation.
  - **Unit tests :** components that have single respondibilities.
    - test_evaluate.py : tests the metricsEval(yValFold, yPred).

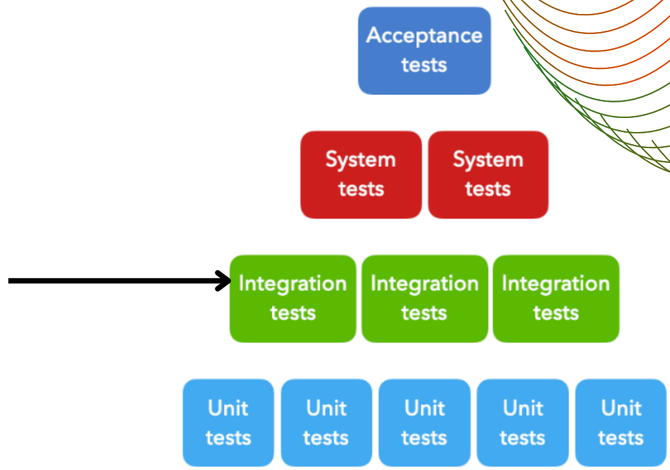
Actual class	Predicted class: Positive	Predicted class: Negative	Total
Positive	True Positive	False Negative	P
Negative	False Positive	True Negative	N




```python
def test_metricsEval():
    yValFold = [1, 0, 1, 1, 0]
    yPred = [1, 0, 0, 1, 1]
    TP = 2
    FN = 1
    P = TP + FN
    TN = 1
    FP = 1
    N = TN + FP
    accuracy = (TP+TN)/(P+N)
    recall = TP / (TP + FN)
    precision = TP / (TP + FP)
    f1 = 2*(precision*recall)/(precision+recall)
    accuracyFct, recallFct, f1Fct, precisionFct = metricsEval(yValFold, yPred)
    assert accuracyFct == accuracy
    assert recall == recallFct
    assert precision == precisionFct
    assert f1 == f1Fct
```

Code testing

- **PyTests:**
  - **Integration tests :** integration of indvidual components.
    - test_dataset_import.py :
      It tests the importBigQuerry() by checking if the imported dataset contains the corresponding column.
    - test_trainingPreprocess.py :
      It tests the trainingPreprocess() by checking if the splitted X and y contain the corresponding column.




```python
def test_dataset_import():
    df = importBigQuerry()
    columns = ["track_id","artists","album_name","track_name","popularity","duration_ms","explicit","danceability",
        "energy","key","loudness","mode","speechiness","acousticness","instrumentalness","liveness","valence",
        "tempo","time_signature","track_genre"]
    for column in columns:
        assert column in df.columns
```

```python
def test_trainingPreprocess():
    df = importBigQuerry()
    X,y = trainingPreprocess(df)
    XColumns = ["duration_ms","danceability","energy","key","loudness","mode",
                "speechiness","acousticness","instrumentalness","liveness","valence","tempo",
                "time_signature"]
    for column in XColumns:
        assert column in X.columns
    assert "hit" in y.name
```

Model pipeline

Transitioning to Orchestrated ML: From Scripts to DAGs

1. Orchestration

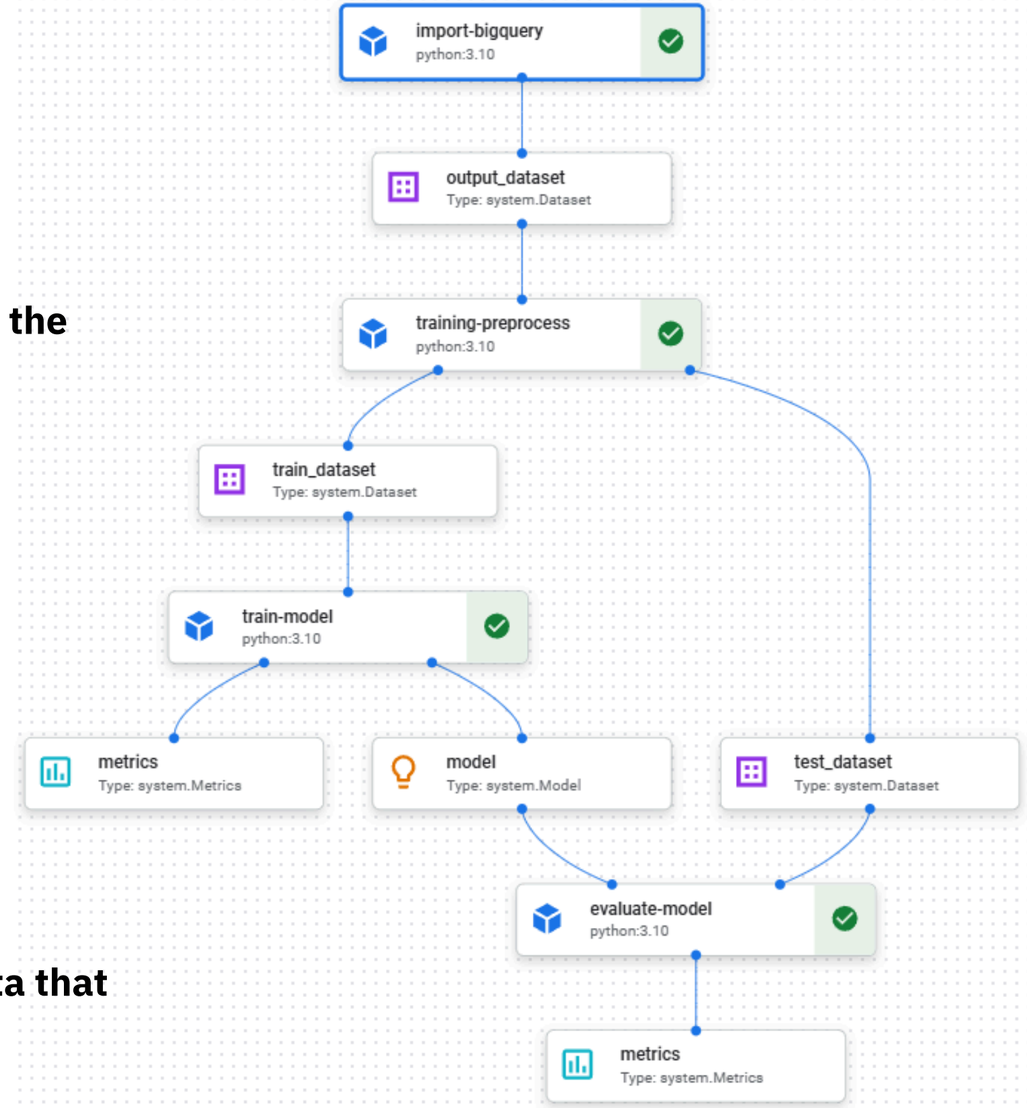
- **Orchestration vs. Execution:** We moved from a manual, linear script to a Directed Acyclic Graph (DAG).
- **System Reliability:** The pipeline is now State-Aware. If a single step fails, the entire process halts safely.

2. Containerized Isolation

- Each box in the graph is a Standalone Docker Container.
- This eliminates the "Works on my machine" problem by freezing the environment in the cloud.

3. Automatic Data Lineage

- Vertex AI tracks every Artifact (the Dataset, the Model, the Metrics).
- We can now trace any model back to the exact version of the BigQuery data that created it.



The Artifact Handshake & Managed Metadata

1. From Session Data → Persistent Data

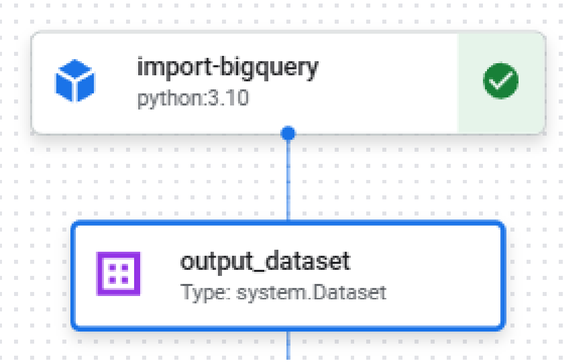
- **Switched from in-memory data to stored artifacts**
- **Each pipeline run creates a timestamped dataset version for reproducibility**

2. Metadata & Auditability

- **Log key metadata (query, schema, row counts)**
- **Full traceability from data source to trained model in Vertex AI**

3. Reduced Data Drift

- **Training uses fixed dataset snapshots**
- **Ensures consistent evaluation and reproducibility**



Artifact Info

Open in ML Metadata

Name	output_dataset
Type	system.Dataset
URI	gs://ai-dj-487610-bucket/pipeline_root/343602206157/ai-dj-training-pipeline-20260330171442/import-bigquery_-7205872103734640 64/output_dataset

Properties

All properties of the artifact.

row_count	114000
source	BigQuery

The Training Component

- **Containerized Isolation (Decorator) :** **Each step runs inside its own container with all required libraries.**
- **Artifact Handshake (Input) :** **Components don’t use fixed file paths anymore.**
- **Experiment Tracking (Metrics) :** **Model performance (like accuracy) is logged, not just printed.**
- **Data Lineage & Metadata (Output) :** **We save more than just the model , we attach useful info (e.g., important features).**

model

Type: system.Model

URI	gs://ai-dj-487610-bucket/pipeline_root/343602206157/ai-dj-training-pipeline-20260330202727/train-model_4280686564100538368/model
Created	March 30, 2026 8:35 PM
Last Modified	March 30, 2026 8:37 PM
top_5_features	["acousticness", "energy", "loudness", "valence", "instrumentalness"]
framework	scikit-learn
columns	["duration_ms", "danceability", "energy", "key", "loudness", "mode", "speechiness", ...
Pipeline runs	ai-dj-training-pipeline-20260330202727

metrics

Type: system.Metrics

URI	gs://ai-dj-487610-bucket/pipeline_root/343602206157/ai-dj-training-pipeline-20260330202727/train-model_4280686564100538368/metrics
Created	March 30, 2026 8:35 PM
Last Modified	March 30, 2026 8:37 PM
training_accuracy	0.9909100877192982
n_estimators	100
Pipeline runs	ai-dj-training-pipeline-20260330202727

Automated Evaluation

Independent Artifact Validation : **The evaluation step loads the actual saved model file, not a temporary variable.**

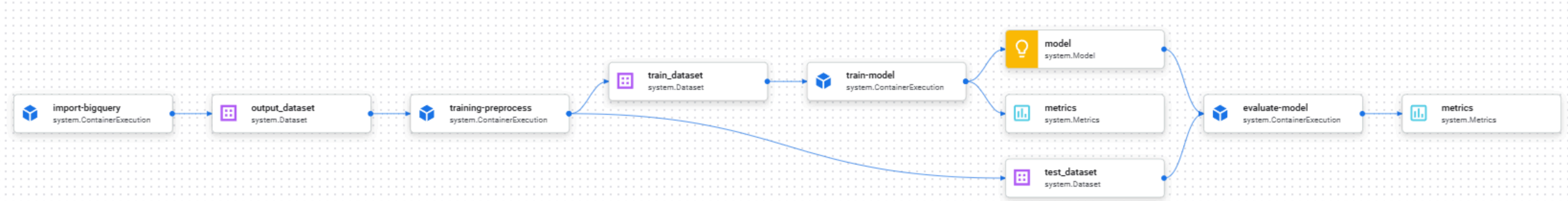
Multi-Dimensional Performance Profiling : **We evaluate using multiple metrics: accuracy, precision, recall, and F1-score.**

Permanent Metadata Archiving : **All metrics are stored in Vertex AI's metadata system.**

metrics

Type: system.Metrics

URI	gs://ai-dj-487610-bucket/pipeline_root/343602206157/ai-dj-training-pipeline-20260330202727/evaluate-model_6586529573314232320/metrics
Created	March 30, 2026 8:37 PM
Last Modified	March 30, 2026 8:38 PM
precision	0.8332052267486548
accuracy	0.9549122807017544
f1_score	0.6783479349186483
recall	0.5720316622691293
Pipeline runs	ai-dj-training-pipeline-20260330202727



Model serving

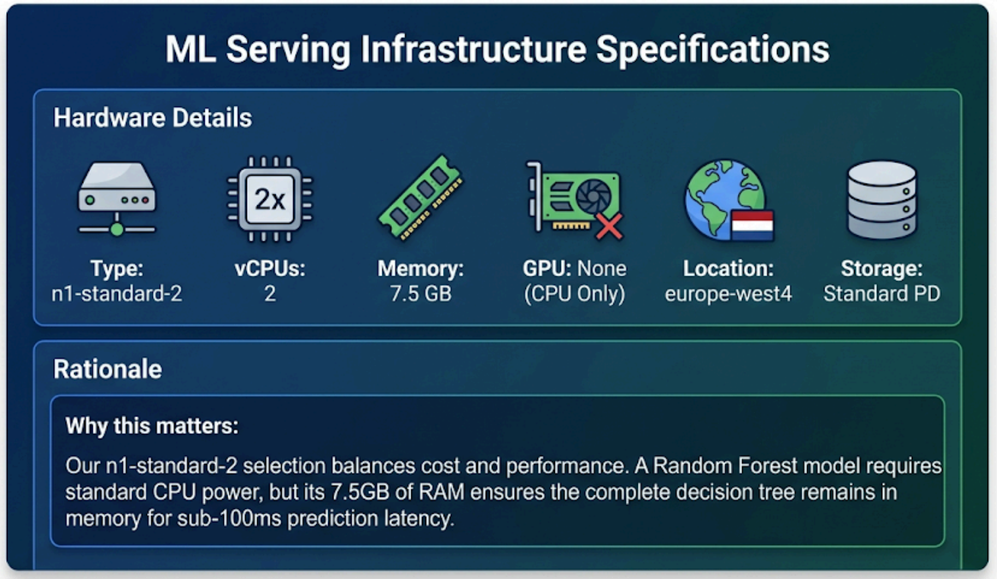
Model Registration & Deployment

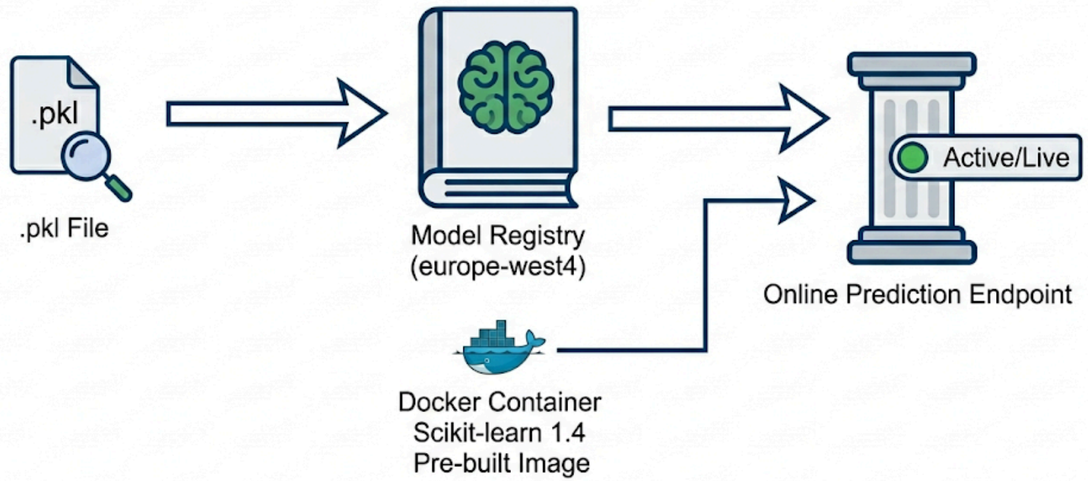
Model Registration : Uploaded the trained model to Vertex AI → creates a versioned source of truth to track updates.

Containerization : Used a pre-built Scikit-learn container → ensures the cloud environment matches training (no version issues).

Online Deployment : Deployed the model to a Vertex AI endpoint (n1-standard-2) → turns the model into a live API.

Result : The model is now a scalable web service accessible via a URI in europe-west4.



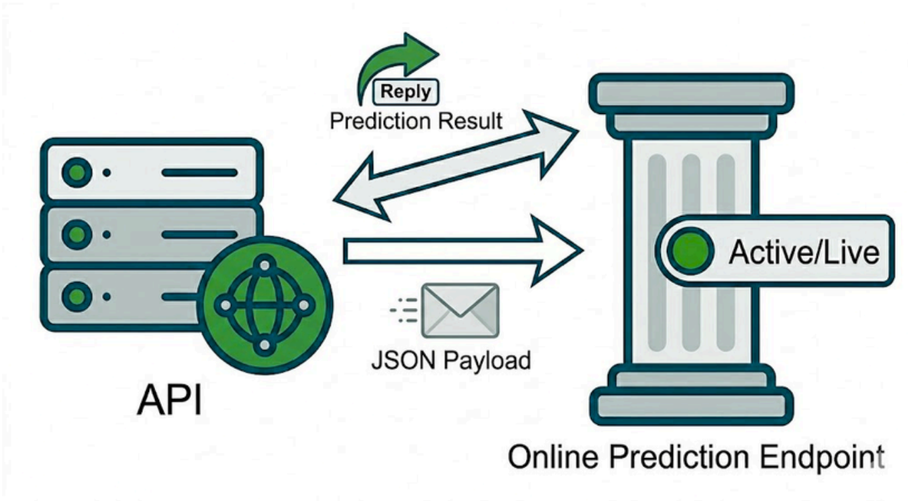


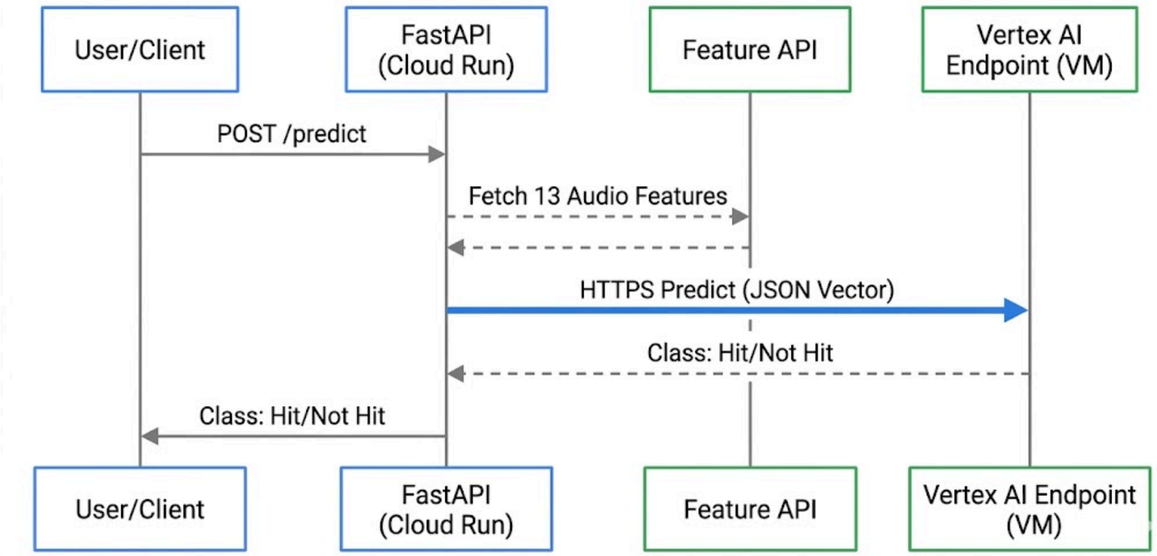
Model Retrieval & Inference Logic

Managed Endpoint: **The API calls a live Vertex AI endpoint instead of using a local model.**

Feature Alignment: **It formats input data into a fixed 13-feature vector to match the model's requirements.**

Serverless Deployment: **Runs on Cloud Run, scales automatically, and sends secure HTTPS requests to get predictions.**





Model deployment

Model deployment

1. REpresentational State Transfer API build

- **FastAPI:**
  - **modern framework, can generate auto-documentation and built-in-swagger (based on OpenAI specification) for easy testing of API endpoints**
  - **asynchronous support, real-time performance and low-latency architecture;**
  - **automatic input validation, uses Pydantic to validate the Spotify data schema**
- **Run locally, in the Docker file container and deployed on Google Cloud Storage**
- **Endpoints: get “health status” / past predictions, post method for predictions**

2. Containerisation with uv

- **Management with uv for fast compilation and dependencies management**
- **Dockerfile build based on Python 3.12 slim (for compatibility and stability)**
- **Modern approach with pyproject.toml (and uv.lock) for automatic management**

3. Cloud deployment

- **Google Cloud Run, on europe-west4: https://ai-dj-api-343602206157.europe-west4.run.app/health**
- **Adaptations required for local version to be deployed**

REST API

1. REST API

- **GET methods : https://ai-dj-api-343602206157.europe-west4.run.app/**
  - **past_predictions**
  - **past_predictions/1 (by id)**

Body Cookies Headers (6) Test Results

{} JSON ▷ Preview Visualize


```
[
    {
        "id": 0,
        "query": "MOCK_FLOP",
        "prediction": "Flop",
        "confidence": "N/A"
    },
    {
        "id": 1,
        "source": "manual_input",
        "prediction": "Flop",
        "class_index": 0,
        "features_received": {
            "danceability": 1,
            "energy": 1,
            "key": 5,
            "loudness": -5.0,
            "mode": 1,
            "speechiness": 1,
            "acousticness": 1,
            "instrumentalness": 0.01,
            "liveness": 1,
            "valence": 1,
            "tempo": 120,
            "duration_ms": 210000,
```

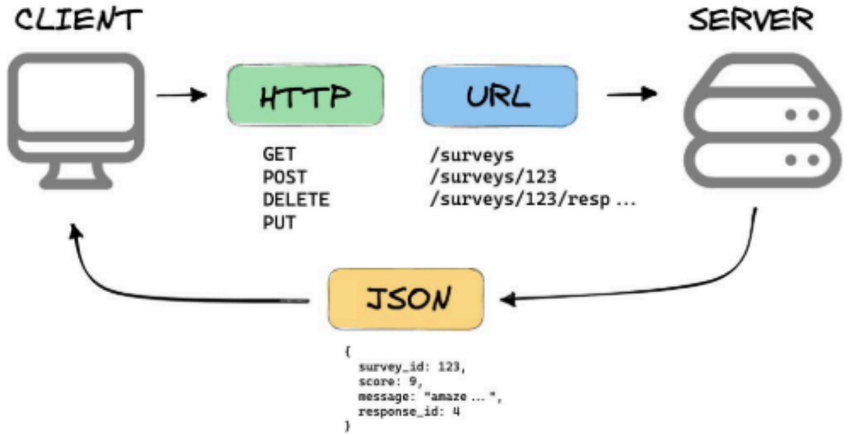

Body Cookies Headers (6) Test Results

{} JSON ▷ Preview Visualize


```
{
    "id": 1,
    "source": "manual_input",
    "prediction": "Flop",
    "class_index": 0,
    "features_received": {
        "danceability": 1,
        "energy": 1,
        "key": 5,
        "loudness": -5.0,
        "mode": 1,
        "speechiness": 1,
        "acousticness": 1,
        "instrumentalness": 0.01,
        "liveness": 1,
        "valence": 1,
        "tempo": 120,
        "duration_ms": 210000,
        "time_signature": 4
    }
}
```


REpresentational State Transfer (REST) API

Roy Fielding's PhD thesis: [Architectural Styles and the Design of Network-based Software Architectures]



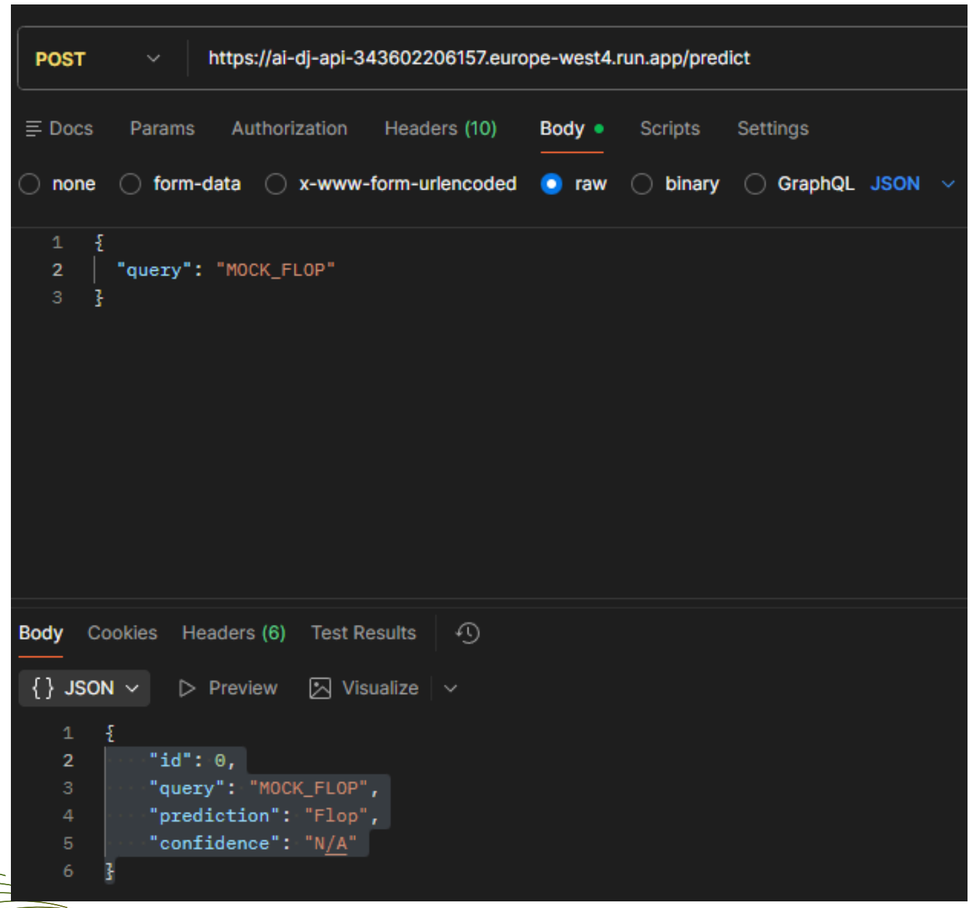
LIÈGE université https://mannhowie.com/rest-api

```python
@app.get("/past_predictions")
def get_history():
    return history


@app.get("/past_predictions/{prediction_id}")
def get_prediction_by_id(prediction_id: int):

    if prediction_id < 0 or prediction_id >= len(history):
        raise HTTPException(status_code=404, detail="Prediction ID not found")

    return history[prediction_id]
```

REST API: POST method

1. REST API

- **POST methods : https://ai-dj-api-343602206157.europe-west4.run.app/**
  - **predict: asynchronous function, query format with a title, prediction features selected (hit or not, confidence)**
  - **predict_manual: asynchronous function, use dictionary of features**




```python
@app.post("/predict")
async def predict(data: dict):
    query = data.get("query")
    if not query:
        raise HTTPException(status_code=400, detail="Query missing")

    # 1. Fetch from Spotify
    audio_features = spotify_service.get_features(query)
    if not audio_features:
        raise HTTPException(status_code=404, detail="Track not found")

    # 2. Get Prediction from Vertex AI
    try:
        result = prediction_service.predict(audio_features)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Vertex AI Error: {str(e)}

    # 3. Format & Store
    response = {
        "id": len(history),
        "query": query,
        "prediction": result["class_name"],
        "confidence": result.get("confidence", "N/A")
    }
    history.append(response)
    return response
```

REST API: POST method

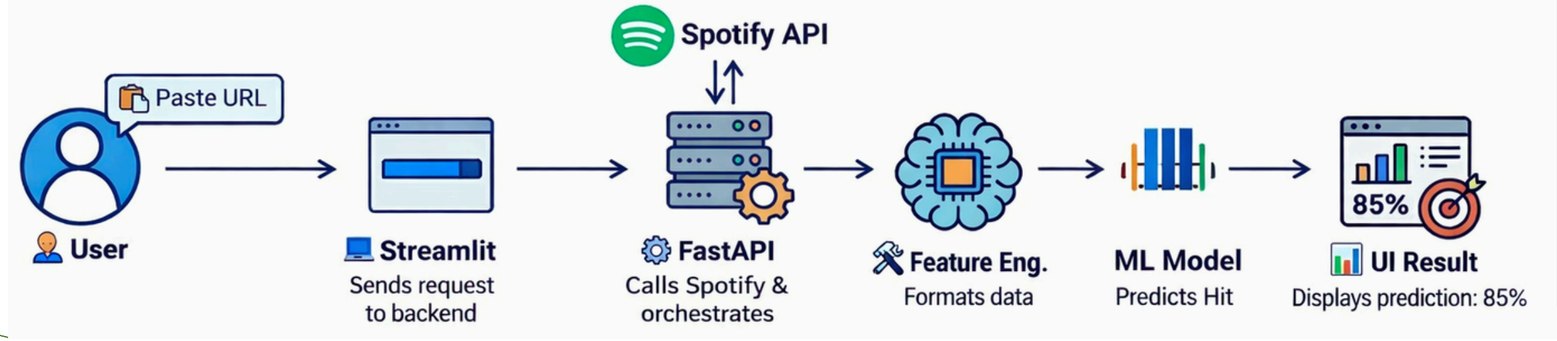
- **Future work:**
  - **re-implement a fancy landing homepage,**

```
@app.get("/")
def home():
    return {"Homepage of Spotify Hit Predictor"}
```

  - **a top-5-features retrieving page (or simple integration in POST predict)**

```
@app.get("/features")
def get_top_features():
    return {"top_5": list_top5}
```

  - **management of url for GET method: submission via Streamlit interface**



POST https://ai-dj-api-343602206157.europe-west4.run.app/predict_manual

Docs Params Authorization Headers (10) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "danceability": 1,
  "energy": 1,
  "key": 5,
  "loudness": -5.0,
  "mode": 1,
  "speechiness": 1,
  "acousticness": 1,
  "instrumentalness": 0.01,
  "liveness": 1,
  "valence":1,
  "tempo": 120,
  "duration_ms": 210000,
  "time_signature": 4
}
```

Body Cookies Headers (6) Test Results

{} JSON Preview Visualize

```
{
    "id": 1,
    "source": "manual_input",
    "prediction": "Flop",
    "class_index": 0,
    "features_received": {
        "danceability": 1,
        "energy": 1,
        "key": 5,
        "loudness": -5.0,
        "mode": 1,
        "speechiness": 1,
        "acousticness": 1,
        "instrumentalness": 0.01,
        "liveness": 1,
        "valence": 1,
        "tempo": 120,
        "duration_ms": 210000
```

Model deployment

2. Containerisation with uv

- **fast compilations: while deploying, required to check the dependencies to manage versions with .toml**
- **Python 3.12 (vs 3.13) for Vertex AI**
- **uv sync. during deployment**
- **port variable for GCS deployment**

pyproject.toml

```toml
[project]
name = "ai-dj"
version = "0.1.0"
description = "Depencies for AI-DJ Project"
readme = "README.md"
requires-python = ">=3.12,<3.14"

dependencies = [
    "fastapi>=0.115.0",
    "uvicorn[standard]>=0.30.0",
    "spotipy>=2.24.0",
    "pandas>=2.2.0",
    "scikit-learn>=1.5.0",
    "numpy>=2.0.0",
    "python-dotenv>=1.0.0",
    "redis>=5.0.0",
    "google-cloud-bigquery>=3.40.1",
    "db-dtypes>=1.5.0",
    "google-cloud-storage>=3.10.1",
    "google-cloud-aiplatform>=1.143.0",
    "kfp>=2.16.0",
]
```

Dockerfile > ...

```dockerfile
# Use 3.12 for maximum stability with Vertex AI libraries
FROM python:3.12-slim

# Install UV
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

# Set working directory
WORKDIR /app

# Copy dependency files first to cache the 'install' layer
COPY pyproject.toml .
# If you have a requirements.txt instead, use: COPY requirements.txt .

# Install dependencies using uv into the system python
RUN uv pip install --system --no-cache fastapi uvicorn google-cloud-aiplatform spotipy python-dotenv pandas

# Copy the rest of the source code
COPY . .

# Cloud Run uses the PORT env variable
EXPOSE 8080

# The CMD needs to point to the file where 'app = FastAPI()' lives
# use the API path we built :
CMD ["sh", "-c", "uvicorn src.api.main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```

Model deployment

3. Cloud deployment

- from the container: needed to fetch models
- used bucket storage for .pkl file to download
- IAM configurations (made during the lab) settings to access the data

src/ui/app.py

```
    # Specify absolute path for Docker image
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
+ BUCKET_NAME = "ai-dj-487610-bucket"
+ MODEL_DIR = os.path.join(BASE_DIR, "../training")
+
+ def download_models():
+     client = storage.Client(project="ai-dj-487610")
+     bucket = client.bucket(BUCKET_NAME)
+     for fname in ["model.pkl", "columns.pkl", "top_5.pkl"]:
+         dest = os.path.join(MODEL_DIR, fname)
+         if not os.path.exists(dest):
+             print(f"Downloading {fname} from GCS...")
+             bucket.blob(f"pkl/{fname}").download_to_filename(dest)
+             print(f"{fname} downloaded.")
+
+ download_models()

    model = pickle.load(open(os.path.join(BASE_DIR, "../training/model.pkl"), "rb"))
    model_columns = pickle.load(open(os.path.join(BASE_DIR, "../training/columns.pkl"), "rb"))
    list_top5 = pickle.load(open(os.path.join(BASE_DIR, "../training/top_5.pkl"), "rb"))

    classes = ["Not a Hit", "Hit"]
    history = []

    def fetch_features_from_spotify(track_input: str):
        if "open.spotify.com" in track_input or "track" in track_input:
            # ID extraction from link
            track_id = track_input.split("/")[-1].split("?")[0]
```

docs > uv-config.txt

```
# Pip compatibility mode, with our requirements.txt
# create a new Python virtual environment
uv venv --python 3.12

# UV configuration
.venv\Scripts\activate # Windows

# install from requirements.txt
uv pip install -r requirements.txt

uv run uvicorn app:app --host 0.0.0.0 --port 8080 --reload


docker build -t ui-app .
docker run -p 8080:8080 ui-app

# Deployment successful
(AI-Dj) PS C:\Users\huiyi\Documents\ULiege\Master\Master-m1_local\AI-Dj> gcloud run deploy ui-app --projec
Building using Dockerfile and deploying container to Cloud Run service [ui-app] in project [ai-dj-487610]
OK Building and deploying... Done.
  OK Validating configuration...
  OK Uploading sources...
  OK Building Container... Logs are available at [https://console.cloud.google.com/cloud-build/builds;regi
  OK Creating Revision...
  OK Routing traffic...
  OK Setting IAM Policy...
Done.
Service [ui-app] revision [ui-app-00003-zq5] has been deployed and is serving 100 percent of traffic.
Service URL: https://ui-app-343602206157.europe-west1.run.app
```

→ Automatisation of Training and Deployment did not require to download model

→ Endpoints are directly connected to the pipeline

Next stage: User interface to show our results...

and deploy it on GCS :)

Streamlit

Q & A